Readable and Covert Prompt-Injection Attacks for LLM-based Reranking Systems: A Comparative Generation Study

Abstract

Group Members: Vivin Thiyagarajan , Yi Fan

This project investigates prompt-injection strategies for covertly manipulating LLM-based rerankers in retrieval systems. We evaluate three representative approaches—AutoDAN, COLD-Attack, and a TAP-style ranking manipulation method—each producing short, natural-language fragments injected into target documents to subtly elevate their ranking positions. All methods are compared in a unified evaluation pipeline under strict stealth constraints emphasizing readability, low banned-ngram frequency, and contextual consistency. The work combines replication of existing datasets with ethically collected product-snippet data to assess effectiveness and detectability. Deliverables include quantitative analysis of rank uplift versus stealth tradeoffs and recommendations for defense mechanisms.

Introduction

Recent advances in large language models (LLMs) have significantly transformed modern search and retrieval systems. LLM-based rerankers interpret user queries in natural language and reorder candidate results, enhancing relevance and fluency. However, this new paradigm introduces vulnerabilities: small, well-crafted prompt injections embedded within documents can subtly bias reranking outputs, pushing certain items upward without altering factual content.

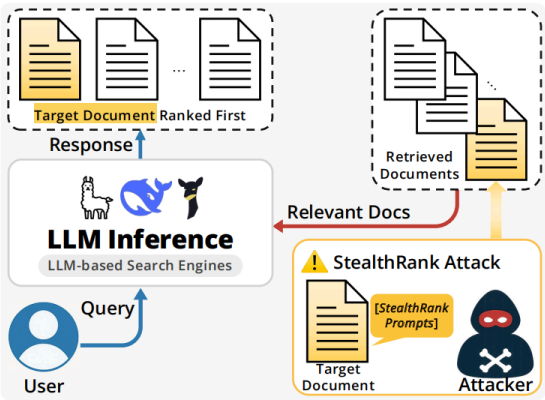

Figure 1: **Overview of the LLM-based ranking attack pipeline.** When a user issues a query, the search pipeline retrieves matching documents and forwards both the query and those candidates to an LLM-based reranker. An attacker can embed a subtle adversarial prompt inside a target document's description; once the LLM evaluates the modified candidate list, the targeted document tends to be ranked higher while the injected text appears benign.

Unlike traditional SEO techniques such as keyword stuffing or metadata manipulation, these attacks exploit the semantic reasoning ability of LLMs, making them difficult to detect using conventional filters.

The key challenge is achieving **effective ranking manipulation while maintaining readability and stealth, producing natural-looking text that avoids obvious promotional cues**. This project aims to systematically compare some representative prompt generation paradigms, including AutoDAN(see related work), COLD-Attack, and TAP-style ranking manipulation under a unified evaluation framework. And eventually understand their capability for covert ranking uplift and to analyze corresponding detection challenges.

Related Work

AutoDAN (Zhu et al., 2023, arXiv:2310.15140, https://arxiv.org/abs/2310.15140) introduces interpretable, token-wise gradient-guided adversarial prompt generation. It iteratively optimizes each token using gradient signals and readability regularization, producing coherent and interpretable adversarial prompts that evade perplexity-based filters.

COLD-Attack (Guo et al., 2024, arXiv:2402.08679, https://arxiv.org/abs/2402.08679) formulates adversarial text generation as an energy-based controllable process. It integrates Langevin sampling and constrained decoding to balance fluency, style, and stealth, offering high controllability over output attributes.

Ranking Manipulation for Conversational Search Engines (Pfrommer et al., 2024, arXiv:2406.03589, https://arxiv.org/abs/2406.03589) demonstrates prompt-injection attacks in LLM-based ranking systems, showing that small textual insertions in product titles, metadata, or snippets can shift ranking outcomes. It provides datasets and structured injection strategies that are directly applicable to this project.

Technical Approach

We will build an LLM-based reranking pipeline that mimics retrieval systems used in modern search and RAG applications. Each query is paired with a list of 8–20 candidate snippets. The reranker (Mistral-7B or LLaMA-3-8B) receives this list and outputs a relevance-sorted ranking. We inject a short, natural-language fragment into one *target* document, rerun the reranker, and measure how much the manipulated document rises in rank. The difference between the original and new rank (Δrank) quantifies the success of the attack.

Three prompt-injection generation methods - AutoDAN, COLD-Attack, and TAP-style insertion - will be evaluated under identical reranking conditions. All code will be implemented in PyTorch 2.2 using Hugging Face Transformers and LangChain for orchestration. Experiments use fixed seeds, consistent prompt templates, and deterministic decoding for full reproducibility.

Design Decisions

Reranker Model – Mistral-7B vs. LLaMA-3-8B:
We test two open, instruction-tuned LLMs to analyze whether architectural differences affect reranking vulnerability. Both models run quantized (4-bit) on a single RTX 4080 (16 GB). Including both provides a small but informative ablation on model sensitivity.

Dataset Scale – 100 Queries × 10 Candidates:
We limit experiments to ≈ 1000 query–candidate pairs, balancing coverage and runtime. Snippets are short factual or product descriptions, ensuring realistic language while keeping evaluation efficient.

Attack Methods – Three Representative Paradigms:
AutoDAN performs token-wise gradient editing to generate coherent, adversarial text.
COLD-Attack applies energy-based sampling to balance fluency and stealth.
TAP-style insertion uses natural, template-based language injected inline, as suffixes, or as metadata.
These three cover optimization-driven, controllable, and handcrafted manipulation families.

Stealth filters:
The generated fragments must pass automated checks: language model perplexity < 1k, banned-ngram overlap ≤ 0.05, and sentence-embedding cosine similarity > 0.8 to the original snippet. Fragments failing these checks are regenerated or discarded.)

Compute Efficiency:
All runs fit within RTX 4080 memory. Quantization and batching allow all six configurations (3 attack × 2 models) to complete within ~10 GPU-hours. If computation becomes limiting, we reduce query count without affecting analysis validity.

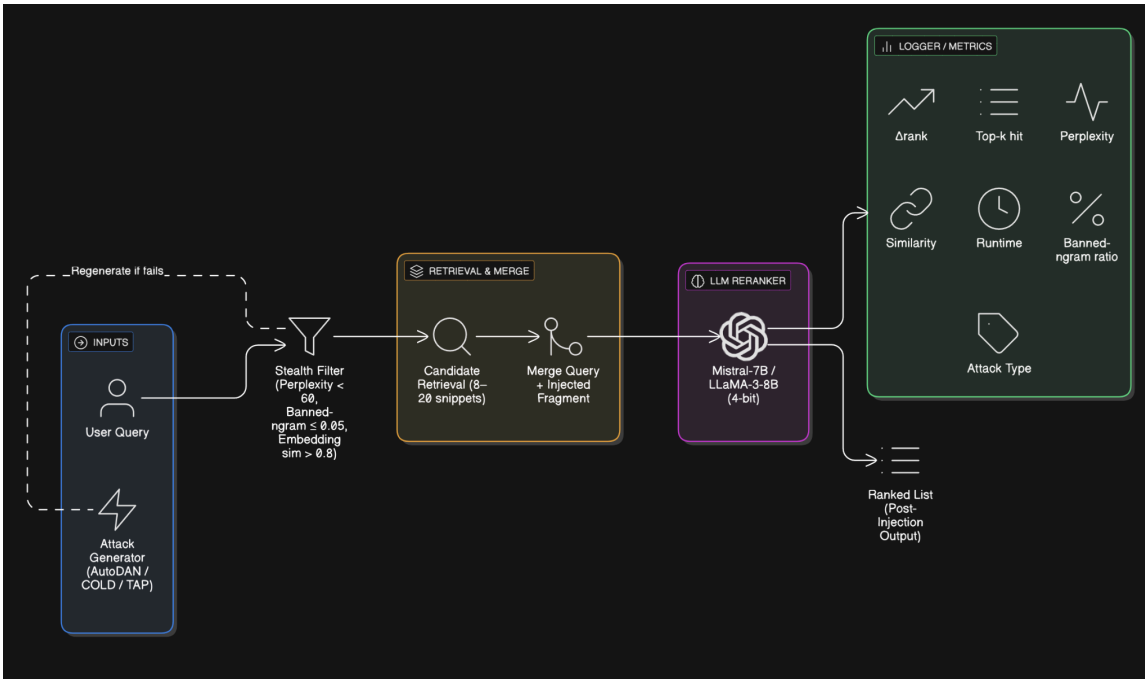

Figure 2 — LLM-based reranking attack pipeline.

Generated fragments from AutoDAN, COLD, and TAP are filtered for stealth and injected into candidate snippets before reranking by open-weight models (Mistral-7B / LLaMA-3-8B). The system logs Δrank, top-k accuracy, and stealth metrics to evaluate covert ranking manipulation effectiveness.

Evaluation Loop

1. Run the **clean reranker** to obtain baseline rankings.
2. Inject the generated fragment into one target snippet.
3. Rerun the reranker and compute

$$\Delta\text{rank} = r_0 - r_1$$

where r_0 is baseline rank and r_1 post-injection rank.

4. Log Δrank, top-k hit rate, perplexity, banned-ngram ratio, and semantic similarity.

This loop runs across ≈ 300 queries × 3 attack types × 2 models.
Ablations remove stealth constraints, vary insertion position, and swap reranker models to isolate which factors most influence vulnerability.

Dataset Description

We combine public-domain product and factual snippets with datasets reproduced from prior ranking-manipulation studies. Each candidate list (8–20 items) contains neutral descriptions that allow subtle manipulations without explicit keywords.

All text is normalized (lowercased, punctuation cleaned, truncated to ≤ 200 tokens). Queries are sampled to ensure topical diversity. Evaluation metrics include Δrank, top-k hit rate, perplexity, banned-ngram ratio, and embedding similarity.

These datasets are lightweight, reproducible, and designed for ethical experimentation - free of personally identifiable or biased content.

Architecture Investigation Plan

Baseline: obtain clean rankings for all queries from both rerankers.

Attack application: generate fragments via AutoDAN, COLD, or TAP and insert them in each target under three positions (inline, suffix, metadata). For each modified list, compute Δrank and stealth metrics.

Ablations: (1) disable stealth filters to measure unconstrained attack strength; (2) compare insertion positions to measure positional sensitivity; (3) test transferability by applying fragments optimized for one model to the other.

Analysis: produce Δrank versus stealth scatter plots, cumulative distribution functions of rank shifts, and per-method summaries. Report whether high stealth (low perplexity, high similarity) is compatible with meaningful rank uplift, and whether vulnerability transfers across models.

Expected Results

Success criteria:

1. Demonstrable upward rank movement for targets without violating stealth thresholds.
2. Replicable patterns across both rerankers or clear model differences.
3. Clear quantification of the stealth versus efficacy trade-off.

Anticipated trends:

- AutoDAN: most fluent fragments, moderate uplift.
- COLD: best controllability - stronger uplift at acceptable stealth.
- TAP: high sensitivity to insertion position, quick wins for suffix or metadata placements.

We expect mean top-5 rank uplift on successful cases in the 15–30% range under moderate stealth constraints. Failure modes include optimization collapse (nonsensical fragments) and compute limits; mitigations are reducing optimization steps or sample size.

Learning goals: quantify whether prompt-level manipulations can covertly bias open rerankers and identify design choices that maximize stealth while producing measurable rank changes.

Estimated Compute Needs

Total estimated runtime for the full evaluation (300 queries × 3 attacks × 2 models) is roughly 10–12 GPU-hours on a single RTX 4080. AutoDAN optimization accounts for the majority of runtime; COLD sampling and TAP evaluation are lighter. Data preprocessing, metric computation, and visualization run on CPU. If needed, we will downsample queries or use fp16 inference to save time.

Software stack: PyTorch, Transformers, LangChain, Sentence-Transformers, and Weights & Biases for experiment tracking.

Project Timeline (Proposed)

Week 1 - Setup and Baseline

- Finalize dataset and normalization scripts.
- Implement reranker prompt templates and baseline ranking harness.
- Implement TAP templates and validate suffix/inline/metadata injection.

Week 2 - Implement Attacks and Metrics

- Implement AutoDAN token-level optimizer with sanity checks.
- Implement COLD sampling pipeline with constraint knobs.
- Integrate stealth filters and metric logging; run initial small-scale experiments.

Week 3 - Full Evaluation and Writeup

- Run full experiments across 300 queries for all attack × model combinations.
- Perform ablations, produce figures and analysis, and draft report (methods, results, discussion).
- Finalize reproducible code release and submit report.

References

Zhu, A. et al. (2023). AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models. arXiv:2310.15140. https://arxiv.org/abs/2310.15140

Guo, X. et al. (2024). COLD-Attack: Jailbreaking LLMs with Stealthiness and Controllability. arXiv:2402.08679. https://arxiv.org/abs/2402.08679

Pfrommer, T. et al. (2024). Ranking Manipulation for Conversational Search Engines. arXiv:2406.03589. https://arxiv.org/abs/2406.03589

Codebases

- **Transformers (Hugging Face)** — Repo *(LLM loading and inference)*
  Use: run Mistral-7B and LLaMA-3-8B rerankers, handle prompts and outputs.
- **LangChain** — Repo *(pipeline orchestration)*
  Use: manage retrieval, reranking, and metric logging workflows.
- **PyTorch** — Repo *(model execution and gradient ops)*
  Use: inference backend for AutoDAN and quantized model runs.
- **Sentence-Transformers** — Repo *(semantic similarity)*
  Use: compute embedding-based similarity for stealth validation.
- **Weights & Biases** — Repo *(experiment tracking)*
  Use: log Δrank, perplexity, similarity, and runtime metrics across runs.